

What is Auxiliary Space = $O(1)$?

Auxiliary Space is the **extra memory used by an algorithm** during its execution, **excluding the memory occupied by the input data**.

Definition

Auxiliary Space is the temporary memory required by an algorithm to solve a problem.

It includes memory used for:

- Temporary variables
- Function call stack (for recursive algorithms)
- Additional arrays or data structures created during execution

It **does not include** the memory used to store the original input.

Auxiliary Space = $O(1)$

When an algorithm uses only a **fixed amount of extra memory**, regardless of the input size, its auxiliary space complexity is **$O(1)$** (Constant Space).

Example: Traversing an Array

```
C
for(i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}
```

Extra Memory Used

- `i` (loop variable)
- `n` (size of array)

Only a few variables are used, no matter whether the array contains:

- 10 elements
- 1,000 elements
- 1,000,000 elements

The extra memory remains the same.

Therefore,

Auxiliary Space = $O(1)$

Example: Linear Search

```
C

int found = 0;

for(i = 0; i < n; i++)
{
    if(a[i] == key)
    {
        found = 1;
        break;
    }
}
```

Extra variables:

- `i`
- `found`
- `key`

Only three variables are used.

Hence,

Auxiliary Space = $O(1)$

Example: Insertion in an Array

```
C

int pos, item, i;

for(i = n-1; i >= pos-1; i--)
{
    a[i+1] = a[i];
}
```

The elements are shifted **within the same array**. No new array is created.

Extra variables:

- `i`
- `pos`
- `item`

Therefore,

Auxiliary Space = $O(1)$

Example: Using an Extra Array

```
C  
  
int b[100];  
  
for(i = 0; i < n; i++)  
{  
    b[i] = a[i];  
}
```

Here, an additional array `b[]` is created.

Extra memory used = `n` elements.

Therefore,

Auxiliary Space = $O(n)$

Comparison

Algorithm	Extra Memory Used	Auxiliary Space
Traversal	Loop variable only	$O(1)$
Linear Search	Few variables	$O(1)$
Array Insertion	Few variables	$O(1)$
Array Deletion	Few variables	$O(1)$
Copy Array	New array of size n	$O(n)$
Merge Sort	Temporary array	$O(n)$
Recursive Factorial	Function call stack	$O(n)$

Time Complexity vs Auxiliary Space

Consider the following code:

```
C  
  
for(i = n-1; i >= pos-1; i--)  
{  
    a[i+1] = a[i];  
}
```

- The loop may execute **n** times, so the **Time Complexity = $O(n)$** .
- However, it uses only one extra variable (**i**) and no additional array, so the **Auxiliary Space = $O(1)$** .

This shows that **time complexity and auxiliary space measure different aspects** of an algorithm:

- **Time Complexity** measures how the running time grows with input size.
- **Auxiliary Space** measures how much **extra memory** the algorithm requires.

Key Points for Exams

- **Auxiliary Space** = Extra memory used by an algorithm (excluding input data).
- **$O(1)$** means the algorithm uses a **constant amount of extra memory**, independent of input size.
- Most basic array operations (traversal, insertion, deletion, searching, updating) use only a few variables, so their **Auxiliary Space is $O(1)$** .
- An algorithm that creates an additional array of size **n** has **Auxiliary Space = $O(n)$** .

Is this conversation helpful so far?